

Ecole Nationale d'Ingénieurs de Carthage Université de Carthage

2ème année Génie Informatique
Programmation des Systèmes Réseaux

SYSTÈME DE MESSAGERIE DISTRIBUÉE "QUANTUM TWIN"

Réalisé par :

Maram HADJ ALI

Douaa BEN MARZOUK
Rached ILEHI

Nour BEN TAHER

Encadré par :

Mme Khaoula BEDOUI

Année universitaire : 2025–2026

Table des matières

1	Contexte et enjeux des systèmes distribués modernes	3
2	Fonctionnement général	5
2.1	Données manipulées	5
2.1.1	Données des clients	5
2.1.2	Données des paires	5
2.1.3	Données d'historique	5
2.2	Opérations	5
3	Phase 1 — Architecture TCP initiale	6
3.1	Présentation	6
3.2	Serveur C — server.c	6
3.3	Client terminal — client.c	7
3.4	Protocole de communication	7
3.5	Diagramme de cas d'utilisation	8
3.6	Communication multi-thread	8
4	Phase 2 — Évolution vers WebSocket, Cloudflare et IA	9
4.1	Limites de la Phase 1 et motivations	9
4.2	Nouvelle architecture en couches	9
4.3	Rôle de chaque composant ajouté	9
4.4	Démarrage de la Phase 2	10
5	Gestion des données	10
6	Visualisation du concept	10
6.1	Intrication quantique et messagerie distribuée	10
6.2	Schéma de messagerie entre jumeaux	11
6.3	Interprétation informatique	11
7	Module d'intelligence artificielle	12
7.1	Présentation	12
7.2	Fonctionnalités	12
7.2.1	Traduction automatique des messages	12
7.2.2	Détection d'anomalies de session	12
7.3	Architecture du module IA	13
8	Conclusion	13
9	Sources	15

Table des figures

1	Évolution du marché des logiciels de collaboration	3
2	Architecture initiale TCP — Phase 1	6
3	Diagramme de cas d'utilisation — Phase 1	8
4	Routage d'un message entre jumeaux avec gestion multi-thread	8
5	Architecture en couches — Phase 2	9
6	Illustration de l'intrication quantique	11
7	Messagerie bidirectionnelle entre jumeaux via le serveur central	11
8	Architecture du module IA — Groq / LLaMA 3	13

Résumé

Ce projet consiste à concevoir un système de messagerie distribuée entre appareils distants, inspiré du concept d'intrication quantique.

Le système a été développé en deux phases successives. Dans un premier temps, un serveur central en C et un client terminal ont été réalisés en utilisant le protocole TCP avec une gestion multi-thread. Dans un second temps, le système a évolué vers une architecture plus ouverte : un proxy WebSocket, un tunnel Cloudflare sécurisé et une interface mobile moderne ont été ajoutés pour permettre l'accès depuis n'importe quel smartphone sur n'importe quel réseau.

Deux clients appariés, appelés *jumeaux numériques*, peuvent s'échanger des messages en temps réel via le serveur central. Un module d'intelligence artificielle (Groq / LLaMA 3) intégré à l'interface mobile permet la traduction automatique des messages et la détection d'anomalies de session.

1 Contexte et enjeux des systèmes distribués modernes

Selon une étude de Grand View Research, le marché mondial des logiciels de collaboration en équipe était estimé à **36 114,2 millions de dollars** en 2024 et devrait atteindre **57 403,8 millions de dollars** d'ici 2030, avec un taux de croissance annuel moyen de **7,4 %**.

Cette croissance s'explique par l'augmentation du travail à distance ainsi que par le besoin croissant de messagerie et de communication en temps réel entre les utilisateurs. Ces tendances justifient pleinement l'intérêt du projet *Quantum Twin*.

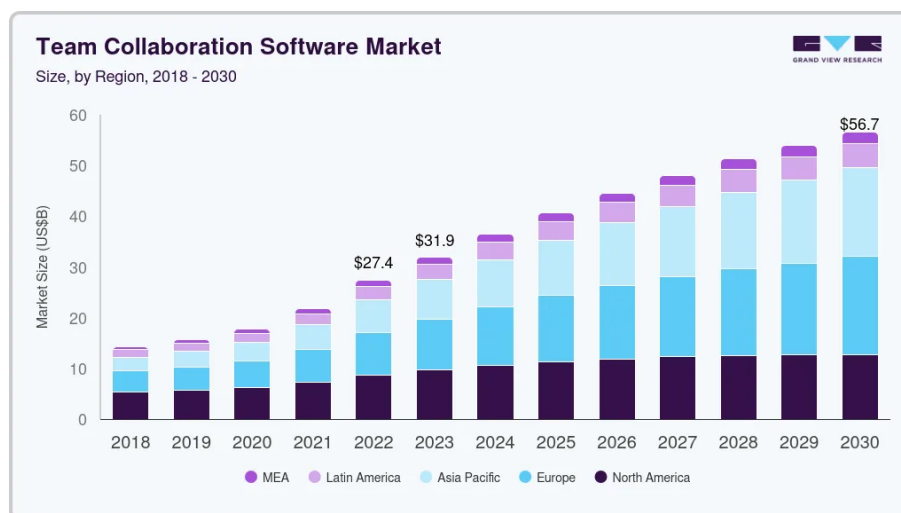


FIGURE 1 – Évolution du marché des logiciels de collaboration



Scannez le code QR pour accéder à l'étude complète

2 Fonctionnement général

Le système repose sur la création de paires de clients. Une fois deux clients appariés (jumeaux), ils peuvent s'échanger des messages en temps réel. Le serveur central reçoit chaque message et le relaie immédiatement vers le destinataire.

2.1 Données manipulées

2.1.1 Données des clients

Chaque client est défini par un identifiant unique attribué à la connexion, un statut (connecté / déconnecté) et un rôle (admin / user).

Fichier `clients.txt` :

```
ID Etat Statut
1  ON  connecte
2  ON  connecte
```

2.1.2 Données des paires

Fichier `pairs.txt` :

```
Client1 Client2
1         2
```

2.1.3 Données d'historique

Fichier `histo.txt` :

Timestamp	ID	Action	Valeur	Résultat
2025-06-01 10:00:01	1	CONNECT	alice	succes
2025-06-01 10:00:05	2	CONNECT	bob	succes
2025-06-01 10:00:10	1	SYNC	2	succes
2025-06-01 10:00:20	1	MSG	"Salut bob !"	envoye
2025-06-01 10:00:25	2	MSG	"Salut alice!"	envoye

2.2 Opérations

Côté client :

- Connexion et authentification au serveur
- Demande d'appariement avec un autre client (SYNC)
- Envoi de messages à son jumeau (MSG)
- Test de connectivité (PING)
- Consultation de la liste des clients (LIST, admin uniquement)

Côté serveur :

- Gestion des connexions multi-thread (un thread par client)
- Authentification via `credentials.txt`
- Routage des messages entre clients (`handle_msg()`)
- Persistance dans les fichiers texte avec protection par mutex
- Historisation de toutes les actions
- Watchdog : redémarrage automatique en cas de crash

3 Phase 1 — Architecture TCP initiale

3.1 Présentation

La première phase du projet repose entièrement sur le protocole **TCP**. Un serveur central écrit en C gère les connexions entrantes et un client terminal interactif permet de s'y connecter depuis un PC ou un appareil Android via Termux. Cette phase constitue le cœur fonctionnel du système : authentification, appariement et messagerie entre jumeaux sont entièrement opérationnels à ce stade.

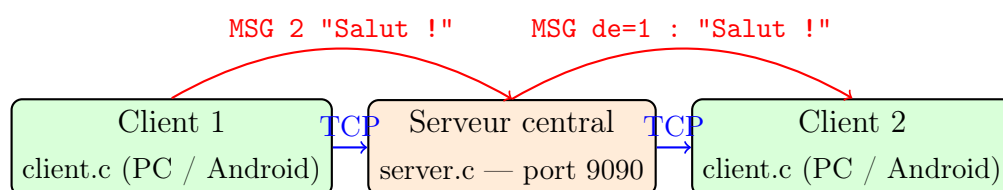


FIGURE 2 – Architecture initiale TCP — Phase 1

3.2 Serveur C — `server.c`

Le serveur est implémenté en C avec les caractéristiques suivantes :

- **Multi-thread** : chaque client accepté génère un thread dédié via `pthread_create()`, permettant de gérer plusieurs connexions simultanées sans blocage.
- **Authentification** : lecture de `credentials.txt`, attribution d'un identifiant unique et d'un rôle (admin / user).
- **Messagerie** : la fonction `handle_msg()` route les messages entre clients via leur socket active.
- **Persistance** : toutes les actions sont enregistrées dans les fichiers texte, protégés par des mutex (`pthread_mutex_t`).
- **Watchdog** : le serveur est lancé via `fork()`. Le processus père surveille le fils et le redémarre après 2 secondes en cas de crash, avec journalisation dans `watchdog.log`.
- **Mode UDP** (bonus) : activé par l'argument `--udp`, illustrant la différence avec TCP.

3.3 Client terminal — client.c

Le client C offre une interface interactive en ligne de commande. Un thread de réception tourne en arrière-plan pour afficher les messages entrants sans bloquer la saisie. Il est compatible PC (Linux/WSL) et Android (Termux).

3.4 Protocole de communication

Commande	Direction	Effet
CONNECT user pass	Client → Serveur	Authentification, attribution d'un ID
SYNC [id]	Client → Serveur	Appariement avec un jumeau
MSG [id] [texte]	Client → Serveur	Envoi d'un message
PING	Client → Serveur	Test de connectivité
LIST	Client → Serveur	Liste clients (admin uniquement)
QUIT	Client → Serveur	Déconnexion propre
MSG de=[id] : [texte]	Serveur → Client	Message reçu
PONG	Serveur → Client	Réponse au PING
OK [message]	Serveur → Client	Confirmation
ERROR [message]	Serveur → Client	Signalement d'une erreur

TABLE 1 – Protocole de communication — Phase 1

3.5 Diagramme de cas d'utilisation

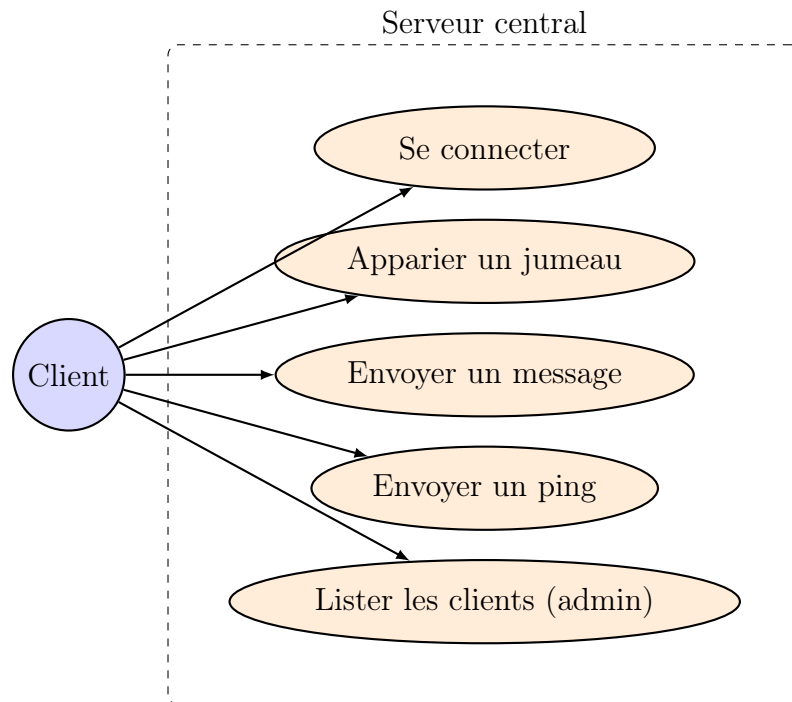


FIGURE 3 – Diagramme de cas d'utilisation — Phase 1

3.6 Communication multi-thread

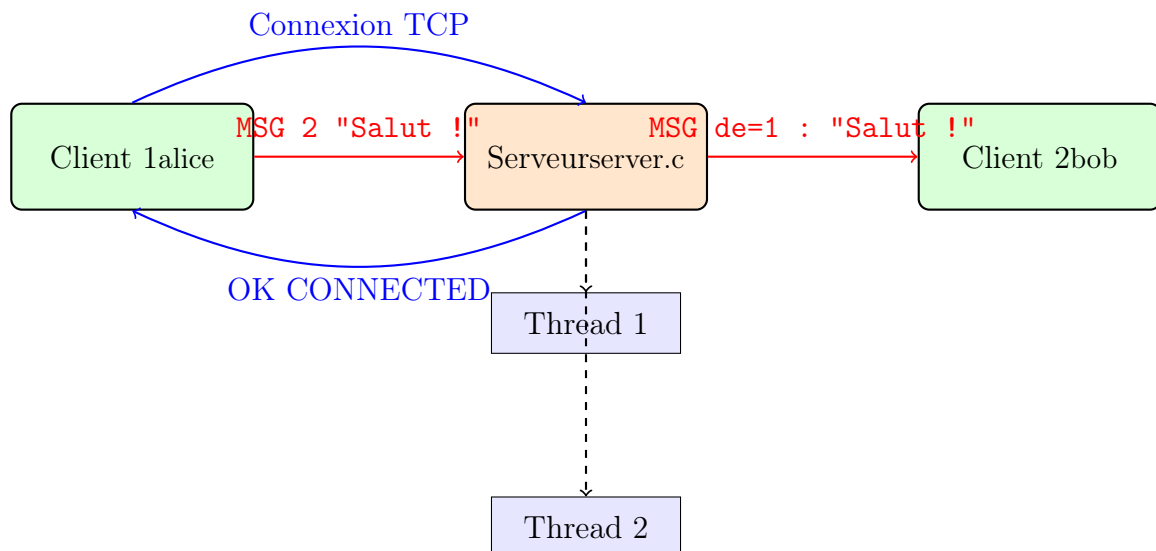


FIGURE 4 – Routage d'un message entre jumeaux avec gestion multi-thread

4 Phase 2 — Évolution vers WebSocket, Cloudflare et IA

4.1 Limites de la Phase 1 et motivations

La Phase 1 fonctionnait parfaitement en réseau local. Cependant, plusieurs obstacles empêchaient l'accès depuis un smartphone sur n'importe quel réseau :

Problème	Cause	Solution adoptée
Navigateur refuse TCP brut	Sécurité du navigateur	proxy.py (WS → TCP)
PC sur WSL	IP interne inaccessible	Tunnel Cloudflare
ngrok bloque WebSocket	Limitation plan gratuit	Remplacé par cloudflared
IP WSL change au redémarrage	WSL dynamique	portproxy Windows

TABLE 2 – Problèmes rencontrés et solutions adoptées en Phase 2

4.2 Nouvelle architecture en couches

La Phase 2 ajoute trois composants entre le smartphone et le serveur C :

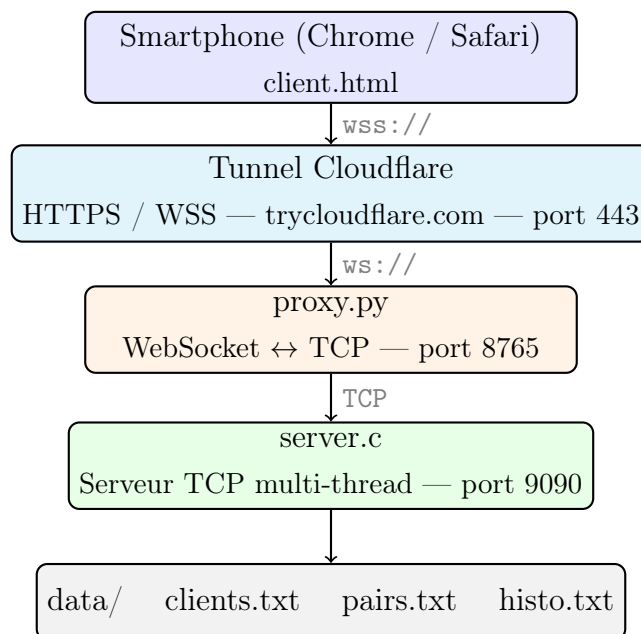


FIGURE 5 – Architecture en couches — Phase 2

4.3 Rôle de chaque composant ajouté

proxy.py est un pont asynchrone écrit en Python (**asyncio** + **websockets**). Il écoute sur le port 8765 en WebSocket et relaie chaque trame vers le serveur C en TCP brut,

et inversement. Le serveur C reste ainsi inchangé tout en devenant accessible depuis un navigateur.

cloudflared expose les ports locaux WSL sur Internet via des URLs publiques sécurisées sans configuration de routeur :

- Port 8765 (proxy) → URL `wss://XXXX.trycloudflare.com`
- Port 8080 (client.html) → URL `https://YYYY.trycloudflare.com`

client.html est l'interface mobile accessible depuis n'importe quel smartphone. Elle est organisée en quatre onglets :

- **TERM** — terminal de commandes brutes
- **MSG** — interface de messagerie avec historique, liste des clients connectés et traduction IA intégrée
- **SYNC** — appariement avec un jumeau
- **AI** — module d'intelligence artificielle

4.4 Démarrage de la Phase 2

Terminal	Commande	Rôle
1	<code>./server 9090</code>	Serveur C TCP
2	<code>python3 proxy.py</code>	Pont WebSocket ↔ TCP
3	<code>python3 -m http.server 8080</code>	Serveur HTTP pour client.html
4	<code>./cloudflared tunnel --url localhost:8765</code>	Tunnel WSS public
5	<code>./cloudflared tunnel --url localhost:8080</code>	Tunnel HTTPS public

TABLE 3 – Les 5 terminaux WSL à lancer à chaque démarrage

5 Gestion des données

Les données sont stockées dans des fichiers texte pour assurer la persistance entre les sessions. Trois fichiers sont utilisés : `clients.txt`, `pairs.txt` et `histo.txt`.

Un mécanisme de verrouillage par mutex (`pthread_mutex_t`) est appliqué sur chaque fichier pour éviter les conflits d'accès concurrents entre les threads. Trois mutex distincts sont définis : `mtx_clients`, `mtx_pairs` et `mtx_histo`.

6 Visualisation du concept

6.1 Intrication quantique et messagerie distribuée

Le concept de ce projet s'inspire du phénomène d'intrication quantique, selon lequel deux particules partagent un lien commun indépendamment de la distance qui les sépare.

Tout événement affectant l'une se répercute instantanément sur l'autre.

Dans notre système, cette idée se traduit par une **messaging entre jumeaux numériques** : deux clients appariés sont liés par un canal de communication direct. Tout message envoyé par l'un est immédiatement transmis à l'autre via le serveur central.

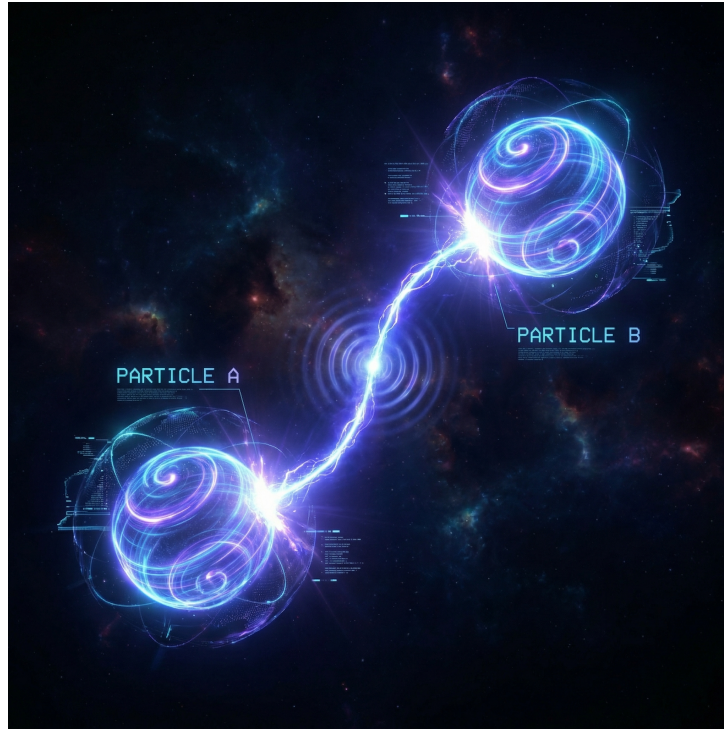


FIGURE 6 – Illustration de l'intrication quantique

Il est important de noter que ce lien repose sur des mécanismes informatiques (TCP, WebSocket, threads) et non sur des phénomènes physiques quantiques réels.

6.2 Schéma de messagerie entre jumeaux

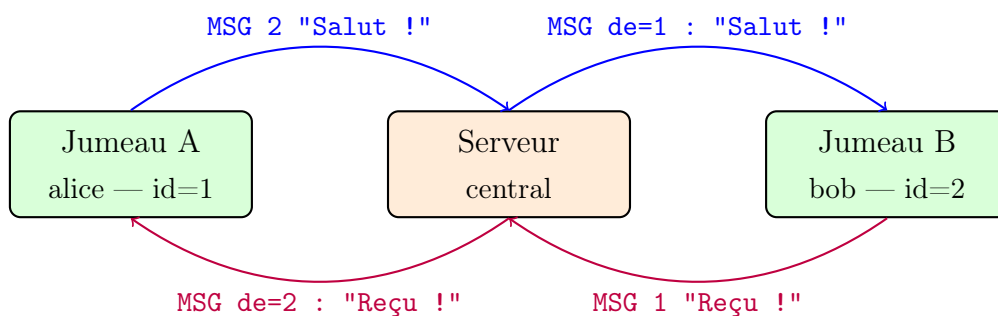


FIGURE 7 – Messagerie bidirectionnelle entre jumeaux via le serveur central

6.3 Interprétation informatique

Lorsqu'un client envoie `MSG [id_cible] [texte]`, la fonction `handle_msg()` du serveur effectue les opérations suivantes :

1. Vérification que le destinataire existe dans la table en mémoire.
2. Récupération de sa socket active (`sockfd`).
3. Construction du message `MSG de=[id] : [texte]`.
4. Envoi immédiat via `send_to_client()`.
5. Enregistrement dans `histo.txt`.

Si le destinataire est hors ligne, le serveur répond `ERROR Destinataire hors-ligne` à l'expéditeur.

7 Module d'intelligence artificielle

7.1 Présentation

L'interface mobile *client.html* intègre un module d'intelligence artificielle accessible depuis l'onglet **AI**. Ce module utilise l'API **Groq** avec le modèle **LLaMA 3.1-8B-Instant**, un grand modèle de langage (LLM) open-source fonctionnant avec une latence très faible grâce à l'infrastructure LPU de Groq.

7.2 Fonctionnalités

7.2.1 Traduction automatique des messages

Lorsqu'un jumeau reçoit un message dans une langue étrangère, il peut le traduire instantanément en cliquant sur le bouton . Cinq langues cibles sont disponibles : français, anglais, arabe, espagnol et allemand.

La traduction est également disponible **en temps réel lors de la saisie** : avant d'envoyer un message, le bouton dans le compositeur active un panneau de traduction qui suggère automatiquement une traduction au fil de la frappe, avec un délai de 900 ms. L'utilisateur peut choisir d'utiliser la traduction proposée ou conserver son texte original.

7.2.2 Détection d'anomalies de session

Le module analyse le journal de session (connexions, messages, erreurs, déconnexions) et génère un rapport structuré. Chaque anomalie est classifiée selon quatre niveaux de sévérité :

Niveau	Indicateur	Exemple
critical		Déconnexions répétées, erreurs d'authentification
warning		Volume élevé de messages, erreurs isolées
info		Informations de session normales
ok		Session saine, aucune anomalie

TABLE 4 – Niveaux de sévérité des anomalies IA

La surveillance peut être activée en mode automatique : l'IA analyse la session toutes les 60 secondes et déclenche une analyse immédiate à chaque nouveau message reçu.

7.3 Architecture du module IA

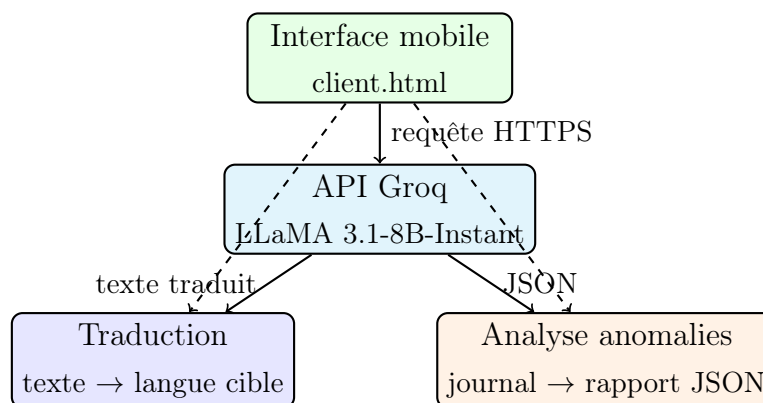


FIGURE 8 – Architecture du module IA — Groq / LLaMA 3

8 Conclusion

Ce projet a suivi une démarche d'évolution progressive en deux phases : une première phase TCP solide servant de socle, puis une seconde phase ouvrant le système au monde via WebSocket, Cloudflare et une interface mobile enrichie d'intelligence artificielle.

Les principaux apports techniques sont :

- Un serveur C multi-thread robuste avec watchdog de redémarrage automatique et mode UDP bonus.
- Un protocole de messagerie simple et extensible (MSG) permettant la communication en temps réel entre jumeaux.
- Une architecture multi-couches (TCP → WebSocket → HTTPS) permettant l'accès depuis un smartphone sur n'importe quel réseau.
- Un module IA intégré offrant traduction en temps réel et surveillance intelligente de la session.

Ce projet ouvre la voie à des extensions futures telles que le chiffrement de bout en bout des messages, la persistance de l'historique des conversations, ou l'intégration de modèles IA plus avancés pour l'analyse comportementale des utilisateurs.

9 Sources

- Grand View Research, *Team Collaboration Software Market* :
<https://www.grandviewresearch.com/industry-analysis/team-collaboration-software-market>
- Groq, *LLaMA 3.1 API Documentation* :
<https://console.groq.com/docs>
- Cloudflare, *Cloudflare Tunnel — Quick Tunnels* :
<https://developers.cloudflare.com/cloudflare-one/connections/connect-networks/>